# Employee Management System (EMS)

## Complete Technical Documentation

Industrial Training Project | Team T-1
October 2026

| Role | Name |
|---|---|
| Team Lead | Teja |
| Team Members | Teja, Ashok, Charan, Kula Shekhar, Jahnavi, Afsar |
| Project Presentation | Teja |
| Project Documentation | Afsar |
| PowerPoint Presentation | Kula Shekhar |
| Development | All team members |

# Document Status

No project source code, repository, or configuration files were present when this document was prepared. Every statement about the system describes the planned design. Nothing here should be read as evidence that a feature is built, tested, or deployed. Sections marked Planned must be verified against the repository before submission.

Items that are unknown or policy-dependent are marked as configurable or to be confirmed, and no organization-specific rule has been invented.

## Table of Contents

*Right-click the field below in Word and choose Update Field to generate page numbers.*

1 Introduction 2 Existing and Proposed Systems 3 SRS 4 System Architecture 5 Portals and Roles 6 Functional Modules 7 Algorithms 8 Database Design 9 API Documentation 10 Security 11 User Interface 12 Deployment and CI/CD 13 Testing 14 Reports 15 Team and Plan 16 Advantages and Future Work 17 Conclusion 18 Viva Preparation

# 1. Introduction

## 1.1 Project Overview

The Employee Management System (EMS) is a multi-tenant platform for managing employee records, attendance, leave, payroll, recruitment, onboarding and offboarding, assets, documents, and reporting. It is delivered as a web application, an Android app, and a Windows desktop app that share one backend.

## 1.2 Background and Motivation

Many small and mid-sized organizations run HR processes across spreadsheets, paper forms, and separate tools. This causes duplicated data, slow approvals, weak audit trails, and inconsistent access control. An integrated system with role-based permissions addresses these gaps.

## 1.3 Problem Statement

Existing HR workflows lack a single source of truth, enforce permissions inconsistently, and make it hard to prove who changed what. The EMS aims to centralize these processes behind one authorization model.

## 1.4 Objectives

- Provide one secure backend shared by web, Android, and Windows clients.
- Enforce access rules in the database through Row Level Security, not only in the user interface.
- Support multiple organizations and branches with strict tenant isolation.
- Record attendance, leave, and payroll with configurable policy rules.
- Keep an audit trail for sensitive changes.

## 1.5 Scope

In scope: the ten portals listed in Chapter 5, the functional modules in Chapter 6, and the three client applications. Out of scope: AI assistants, retrieval-augmented generation, and chatbots. Tax filing, bank transfers, and biometric hardware integration are not planned.

## 1.6 Intended Users

Super administrators, organization administrators, HR staff, recruiters, finance and payroll staff, managers, department heads, IT asset staff, employees, and auditors.

## 1.7 Limitations

- Payroll formulas depend on each organization's policy and must be configured.
- Offline operation on Android and Windows is not planned for the first release.
- Performance and load targets have not been measured yet.

# 2. Existing and Proposed Systems

## 2.1 Existing System

Typical current practice (general description, not a survey of a specific organization): spreadsheets for attendance and payroll, email for leave requests, paper or PDF forms for onboarding, and a shared drive for documents without version control.

## 2.2 Problems in the Existing System

- Duplicated data entry across spreadsheets.
- No consistent permission model; sensitive salary data is often visible to too many people.
- Approvals happen by email with no recorded state.
- No reliable audit history.
- Difficult to report across branches.

## 2.3 Proposed System

A single platform with role-based portals, approval workflows stored as records, database-enforced access rules, and a shared API used by all clients.

## 2.4 Existing versus Proposed Comparison

| Aspect | Existing | Proposed (planned) |
|---|---|---|
| Data | Spreadsheets, multiple copies | One database per deployment, tenant-scoped |
| Approvals | Email threads | Request records with status and history |
| Access control | Informal, file-based | RBAC plus PostgreSQL RLS |
| Auditing | Limited | Audit log for sensitive actions |
| Clients | Desktop files | Web, Android, Windows over one API |

**Status:** *The rows above describe intended behavior. They are not confirmed features until tested.*

# 3. Software Requirements Specification

## 3.1 Functional Requirements (planned)

- FR-01 Users sign in with Supabase Auth.
- FR-02 Administrators create organizations, branches, and departments.
- FR-03 HR staff create and update employee records.
- FR-04 Employees record attendance and submit leave requests.
- FR-05 Managers approve or reject leave within their reporting line.
- FR-06 Finance staff generate payroll runs and submit them for approval.
- FR-07 Recruiters track candidates through pipeline stages.
- FR-08 Users upload and download documents subject to permissions.
- FR-09 Auditors view audit logs without edit rights.

## 3.2 Non-Functional Requirements (targets, not yet measured)

| ID | Area | Target |
|---|---|---|
| NFR-01 | Security | No cross-tenant read or write, verified by tests |
| NFR-02 | Availability | To be agreed with the client |
| NFR-03 | Response time | To be measured under agreed load |
| NFR-04 | Accessibility | Keyboard navigation and sufficient contrast on web |
| NFR-05 | Maintainability | Typed code (TypeScript), migrations under version control |

## 3.3 Hardware Requirements

- Developer machine: a modern laptop capable of running Node.js and an Android emulator.
- Android device: Android version to be confirmed against the Expo target.
- Windows device: Windows version to be confirmed against the Tauri build target.

## 3.4 Software Requirements

| Layer | Technology |
|---|---|
| Web frontend | Next.js, React, TypeScript, Tailwind CSS |
| Android | React Native with Expo |
| Windows | Tauri with React |
| Backend platform | Supabase (PostgreSQL, Auth, Storage, Realtime) |
| Hosting | Vercel |
| Source control and CI | GitHub |
| Monorepo | pnpm workspaces |

## 3.5 Constraints, Assumptions, and Dependencies

- Assumption: each organization supplies its own leave and payroll policies.
- Dependency: Supabase and Vercel service availability and quotas.

- Constraint: the service-role key must never ship in a client application.

## 3.6 Sample Use Case and Acceptance Criteria

UC-LEAVE-01 Employee submits a leave request. Acceptance: the request is stored with status Pending, the manager is notified, and the employee cannot see other employees' requests.

# 4. System Architecture

## 4.1 Overall Architecture

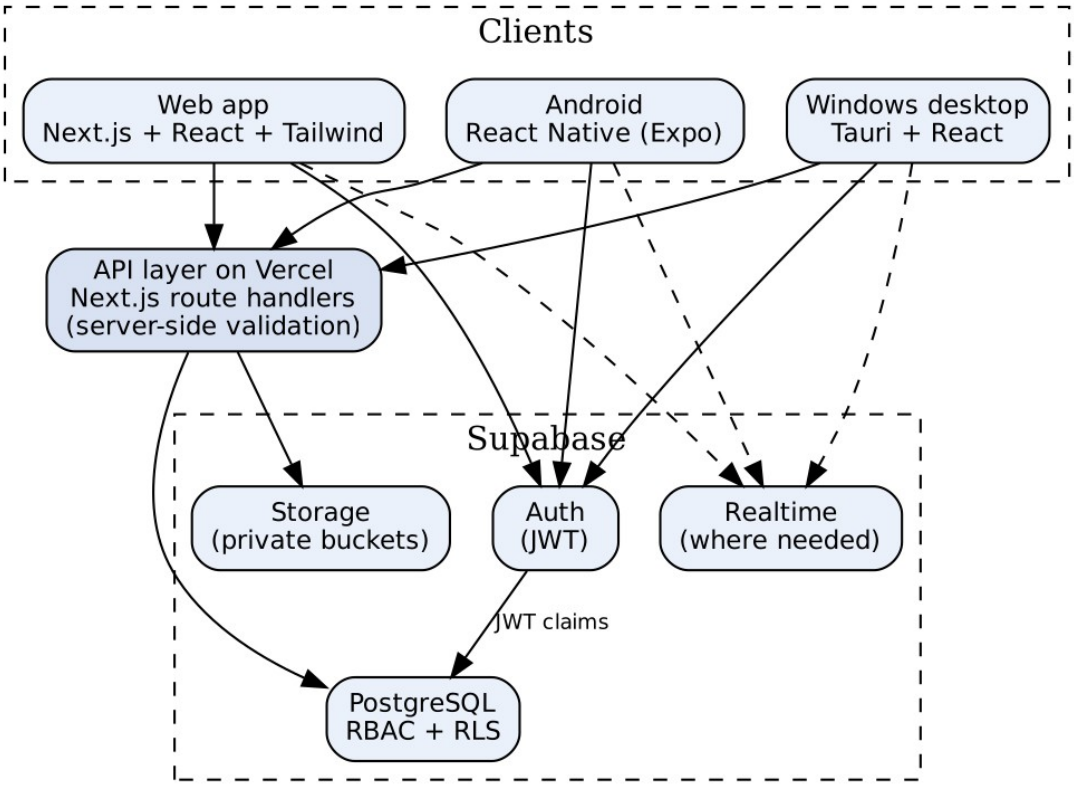



**Figure 4.1. Overall system architecture**

Three clients (Next.js web app on Vercel, Expo Android app, Tauri Windows app) call a common API layer. Data and identity live in Supabase. Authorization is enforced in PostgreSQL through RBAC helper functions and Row Level Security policies, so every client receives the same rules.

## 4.2 Client Architecture

- Web: Next.js App Router, server components for data reads, client components for interaction.
- Android: Expo app using the Supabase JavaScript client with secure session storage.
- Windows: Tauri shell embedding a React UI that uses the same Supabase client.

## 4.3 Vercel Application and API Layer

Next.js route handlers on Vercel perform server-side validation for operations that need more than a direct table write, such as payroll runs and document uploads. The service-role key is available only to these server functions as an environment variable.

## 4.4 Supabase Services

- Auth: email sign-in and password recovery.
- PostgreSQL: all business data, with RLS enabled on every tenant table.

- Storage: private buckets, accessed through signed URLs.
- Realtime: used only where live updates are needed, such as notification counts.

## 4.5 Request and Authentication Flow

1. The user signs in through Supabase Auth and receives a JWT. 2. The client sends the JWT with each request. 3. PostgreSQL reads the JWT claims and evaluates RLS policies using the user's organization and role. 4. Only permitted rows are returned or changed.

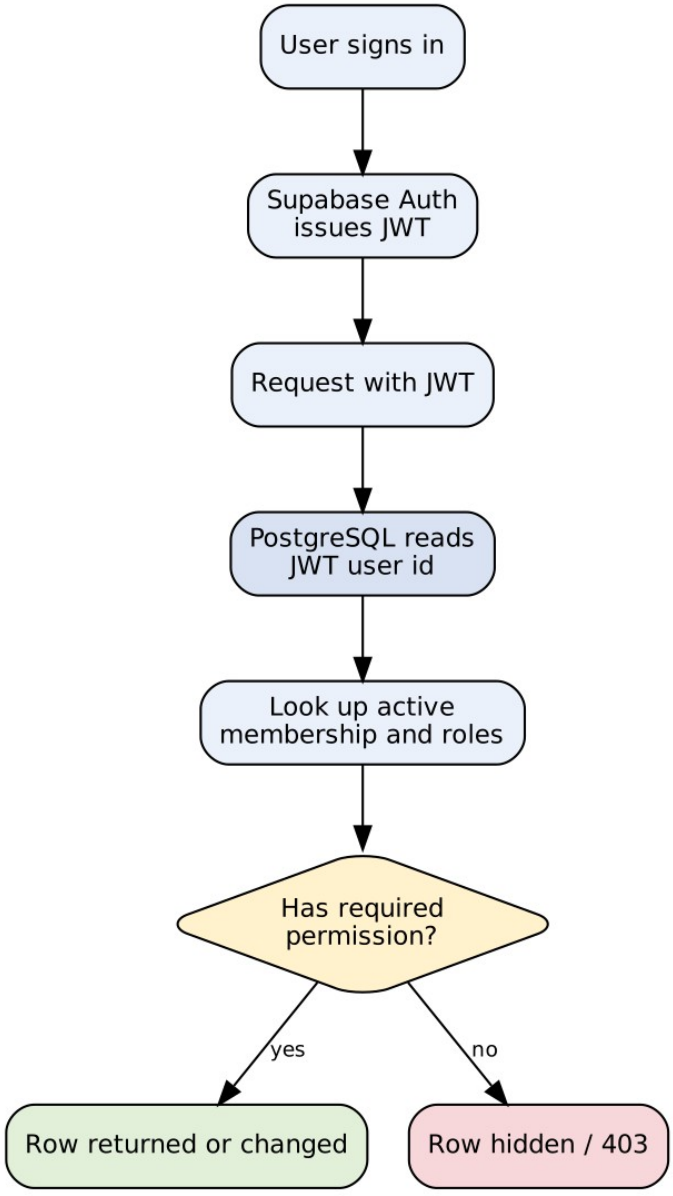



**Figure 4.2. Authentication and authorization flow**

## 4.6 Data Flow Diagram (conceptual)

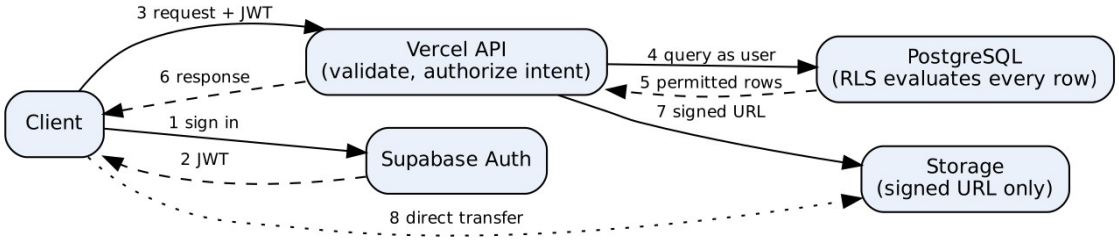



**Figure 4.3. Client-to-backend data flow**

# 5. Portals and User Roles

Each portal is a set of screens restricted by role. The table below summarizes purpose and typical users. Permissions are enforced by RLS; the portal only controls what is shown.

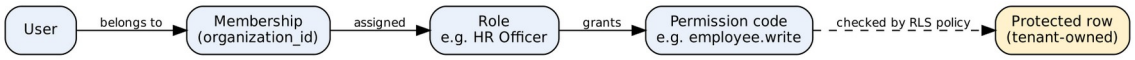



**Figure 5.1. Role and permission model**

| Portal | Purpose | Typical users |
|---|---|---|
| Super Admin | Platform setup across organizations | Platform operators |
| Organization Admin | Organization settings, branches, roles | Company administrators |
| HR | Employee records, lifecycle, documents | HR staff |
| Recruitment | Job postings and candidate pipeline | Recruiters |
| Finance and Payroll | Salary structures, payroll runs | Payroll staff |
| Manager | Approve team leave and requests | Line managers |
| Department Head | Department reports and headcount | Department heads |
| IT and Assets | Asset register and asset requests | IT staff |
| Employee | Own profile, attendance, leave, payslips | All employees |
| Auditor | Read-only audit and compliance views | Internal or external auditors |

**Status:** *Per-portal feature lists, inputs, outputs, and workflows will be expanded during documentation review. Permissions must be checked against the actual RLS policies.*

# 6. Functional Modules

Each module is listed with its purpose and primary validation rules. Detailed workflows follow the same pattern: input, processing, output, roles, errors.




**Figure 6.1. Employee lifecycle**

| Module | Purpose | Key validation rules |
|---|---|---|
| Authentication and users | Sign-in and account lifecycle | Unique email per organization; password policy set in Supabase |
| Organization and departments | Structure of company | Department codes unique within organization |
| Employee management | Employee master data | Employee ID unique per organization; mandatory name and joining date |
| Attendance and shifts | Clock in and out, shift assignment | No overlapping open sessions per employee |
| Leave | Requests, balances, approval | Balance must cover request; no overlapping approved leave |
| Payroll | Salary computation and approval | Locked after approval; changes need a new run |
| Recruitment | Candidate pipeline | Stage moves follow allowed transitions |
| Onboarding and offboarding | Checklists for joiners and leavers | Mandatory items must be complete before closing |
| Tasks and performance | Goals and reviews | Review cycle dates must not overlap |
| Assets | Issue and return of equipment | An asset has at most one active assignment |
| Requests and approvals | Generic approval records | Only the designated approver can act |
| Documents | Upload, versioning, secure download | File type and size limits; private bucket |
| Announcements and notifications | Company messages and alerts | Scoped to organization or department |
| Reports and analytics | Aggregated views | Sensitive reports require explicit permission |
| Audit and security | Record of sensitive changes | Append-only from the application |

**Status:** *Validation rules above are proposed. Confirm each against the implemented schema.*

# 7. Algorithms and Business Logic

## 7.1 Employee ID Generation

Pseudocode: take the organization's configured prefix, append a zero-padded sequence per organization, and increment inside a transaction to avoid duplicates. The prefix and padding length are configurable.

## 7.2 Role and Permission Checking

Pseudocode: resolve the user's active role assignments for the organization; the permission is granted if any assigned role contains the requested permission code; otherwise deny. The same check runs inside RLS policy functions.

## 7.3 Attendance Duration

Pseudocode: duration = clock_out - clock_in, minus configured break minutes. Open sessions are excluded from totals until closed.

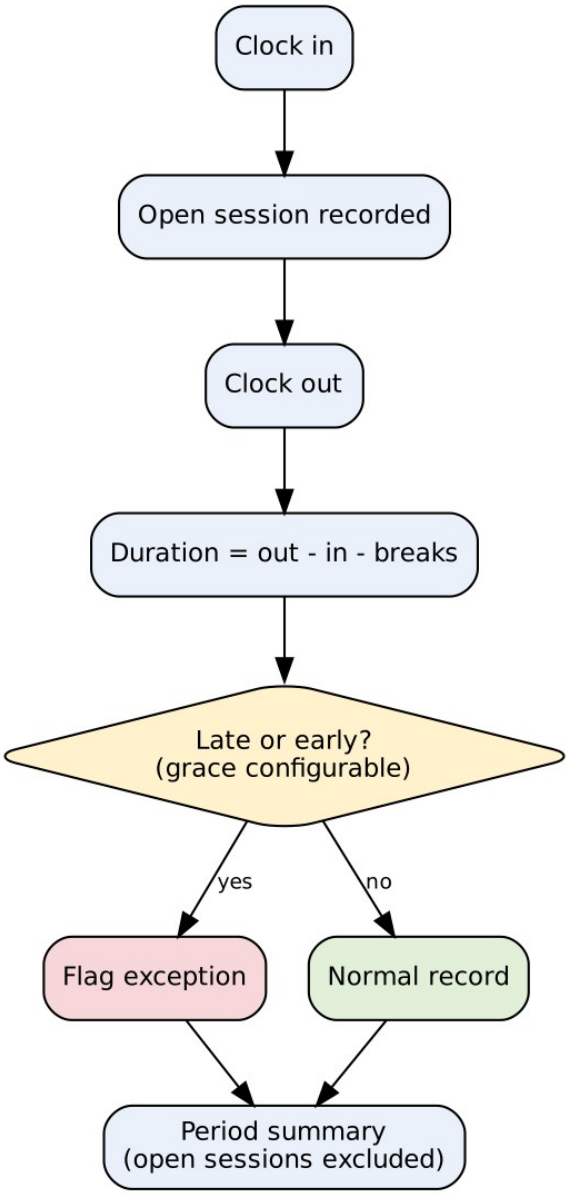



**Figure 7.1. Attendance workflow**

## 7.4 Late Arrival and Early Departure

Pseudocode: late if clock_in > shift_start + grace_minutes; early if clock_out < shift_end - grace_minutes. Grace minutes are configurable and not fixed by this document.

## 7.5 Overtime

Policy-dependent. The overtime threshold, multiplier, and whether it is daily or weekly are configured per organization. No default rule is assumed here.

## 7.6 Leave Balance and Approval

Balance = accrued + carried_forward - used - pending. A request is allowed when requested_days <= balance. Approval moves the request from Pending to Approved or Rejected and writes a history row; the balance is recalculated from ledger entries.

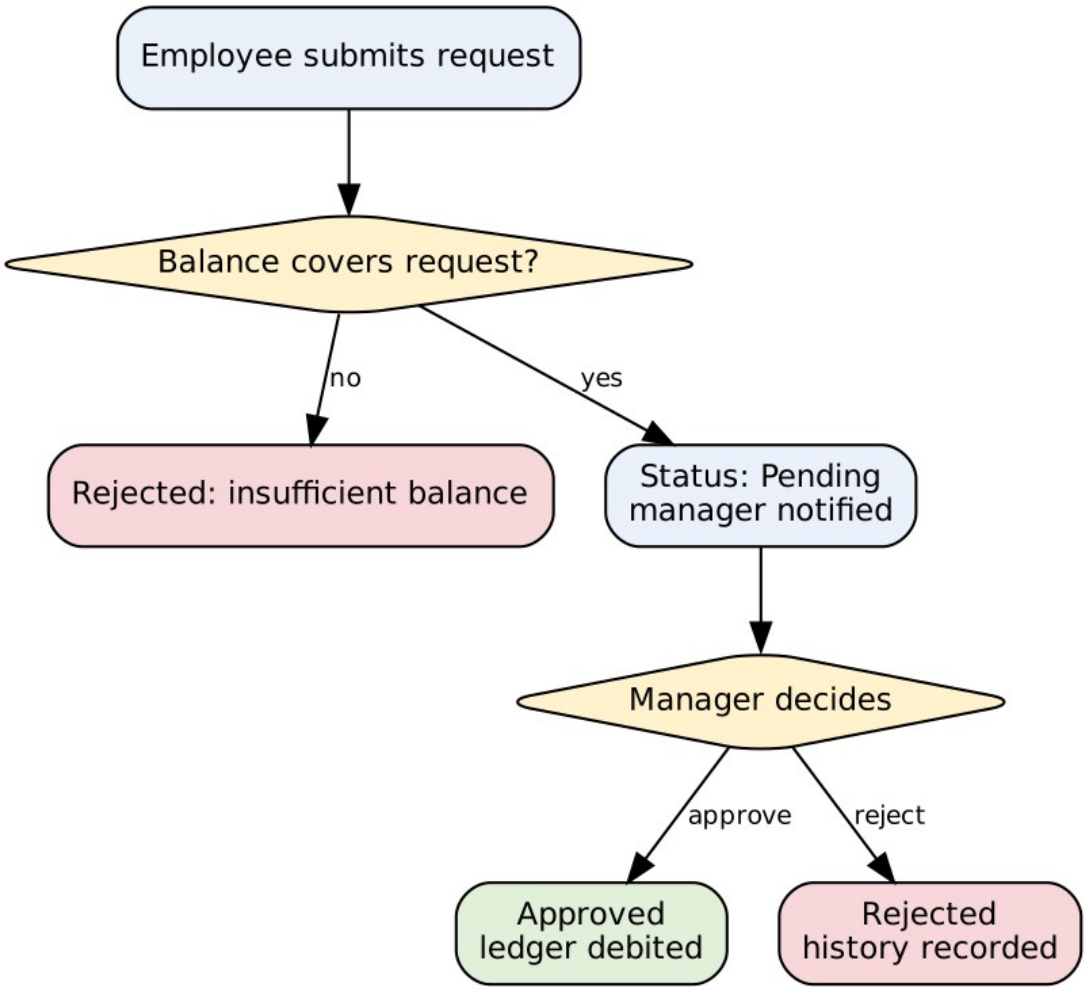



**Figure 7.2. Leave approval workflow**

## 7.7 Gross and Net Salary

Gross = base + configured allowances. Net = gross - configured deductions. Tax and statutory deductions are inputs from the organization's configured tables and are not computed by a fixed formula in this document.

## 7.8 Payroll Processing and Approval

Steps: create run for period, compute each employee's gross and net, lock inputs, submit for approval, approve, then freeze. Corrections create a new run rather than editing an approved one.

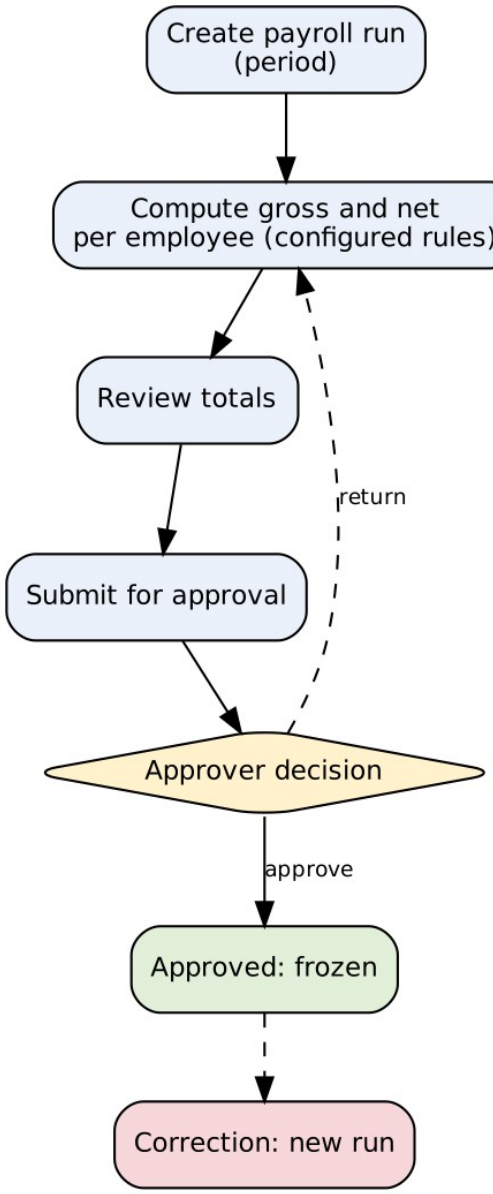



**Figure 7.3. Payroll processing and approval workflow**

## 7.9 Recruitment Pipeline

Stages: Applied, Screening, Interview, Offer, Hired, Rejected. Transitions are restricted by a configured map, for example Applied cannot jump directly to Offer.

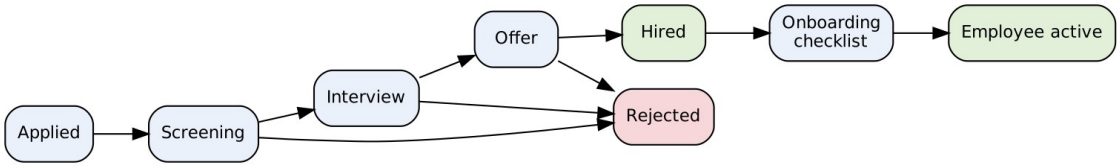



**Figure 7.4. Recruitment and onboarding workflow**

## 7.10 Onboarding and Offboarding

Each checklist item has an owner and status. The employee record becomes active when mandatory onboarding items are complete; offboarding deactivates access and triggers asset return items.

## 7.11 Report Aggregation and Audit Recording

Reports aggregate from base tables with filters for organization, branch, and period. Audit rows record actor, action, entity, timestamp, and before and after values for sensitive tables.

# 8. Database Design

The schema uses UUID primary keys, timestamptz for time values, numeric types for money, and organization_id on every tenant-owned table. Tenant isolation is enforced by RLS on each table.

## 8.1 Core Tables (proposed)

| Table | Purpose | Key columns | Keys and constraints |
|---|---|---|---|
| organizations | Tenant root | id, name, created_at | PK id |
| branches | Organization sites | id, organization_id, name | FK organization_id |
| departments | Org units | id, organization_id, code, name | Unique (organization_id, code) |
| designations | Job titles | id, organization_id, title | FK organization_id |
| profiles | Maps auth user to employee | id (= auth.users.id), organization_id | FK auth.users |
| employees | Employee master | id, organization_id, employee_code, department_id, designation_id, joined_on | Unique (organization_id, employee_code) |
| employee_history | Changes over time | id, employee_id, changed_at, field, old_value, new_value | FK employee_id |
| roles | Named roles | id, organization_id, name | Unique (organization_id, name) |
| permissions | Permission codes | id, code | Unique code |
| role_permissions | Role to permission | role_id, permission_id | Composite PK |
| user_roles | User role assignments | user_id, organization_id, role_id | Composite PK |
| attendance_records | Clock sessions | id, organization_id, employee_id, clock_in, clock_out | Index (employee_id, clock_in) |
| shifts | Shift definitions | id, organization_id, start_time, end_time | Check end > start |
| leave_types | Leave categories | id, organization_id, name, accrual_rule | Unique (organization_id, name) |
| leave_requests | Requests | id, organization_id, employee_id, leave_type_id, starts_on, ends_on, status | Check ends_on >= starts_on |
| leave_ledger | Balance movements | id, organization_id, employee_id, leave_type_id, delta, reason | Balance derived from ledger |
| salary_structures | Pay components | id, organization_id, employee_id, component, amount | numeric(12,2) |
| payroll_runs | Payroll cycles | id, organization_id, period, status | Unique (organization_id, period) |
| payroll_records | Per-employee pay | id, run_id, employee_id, gross, net | numeric(12,2) |
| candidates | Recruitment | id, organization_id, name, stage | Stage check constraint |
| onboarding_items | Checklist items | id, organization_id, employee_id, title, done_at | FK employee_id |

| Table | Purpose | Key columns | Keys and constraints |
|---|---|---|---|
| assets | Equipment | id, organization_id, code, status | Unique (organization_id, code) |
| asset_assignments | Who holds what | id, asset_id, employee_id, assigned_on, returned_on | Partial unique index on active rows |
| documents | Files | id, organization_id, owner_id, storage_path, current_version | FK organization_id |
| document_versions | Version history | id, document_id, version_no, storage_path | Unique (document_id, version_no) |
| notifications | Alerts | id, organization_id, user_id, read_at | Index (user_id, read_at) |
| announcements | Messages | id, organization_id, scope, body, published_at | Scope check constraint |
| audit_logs | Change record | id, organization_id, actor_id, action, entity, occurred_at | Append-only policy |
| settings | Per-organization config | organization_id, key, value | PK (organization_id, key) |

**Status:** *This is a proposed schema. Column lists must be checked against the actual migration files before submission.*

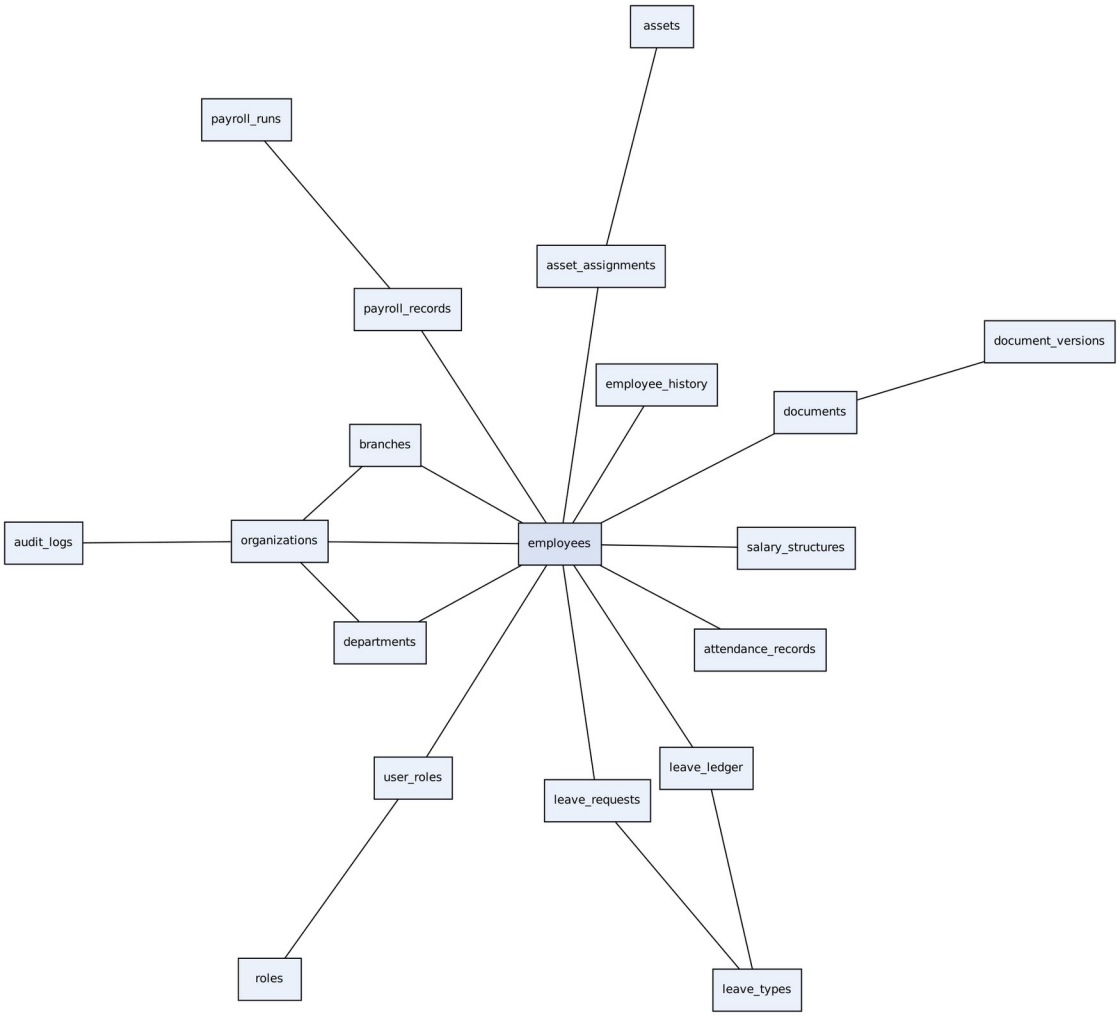



**Figure 8.1. Database entity-relationship diagram**

Relationships: every tenant table links to organizations, so the organization is the root of all isolation. Employees are central: they belong to a department and branch, hold role assignments, and own attendance, leave, salary, payroll, asset, and document records. Leave balances are derived from the leave ledger rather than stored as a single mutable number. Payroll records reference both a run and an employee. Documents keep a version table so earlier files stay traceable.

## 8.2 Tenant Isolation

Every table with organization_id has an RLS policy that allows reads and writes only when the row's organization_id matches the organization in the caller's active membership, and the required permission is held. Cross-tenant tests are listed in Chapter 13.

# 9. API Documentation

The endpoints below are proposed. Each must be verified against the implemented route handlers before it is documented as available.

| Method | Path | Purpose | Permission |
|---|---|---|---|
| GET | /api/me | Current user and organization context | Authenticated |
| GET | /api/employees | List employees in scope | employee.read |
| POST | /api/employees | Create employee | employee.write |
| PATCH | /api/employees/{id} | Update employee | employee.write |
| POST | /api/attendance/clock | Clock in or out | attendance.self |
| GET | /api/attendance/summary | Attendance totals for period | attendance.read |
| POST | /api/leave/requests | Submit leave request | leave.self |
| POST | /api/leave/requests/{id}/decision | Approve or reject | leave.approve |
| POST | /api/payroll/runs | Create payroll run | payroll.write |
| POST | /api/payroll/runs/{id}/approve | Approve run | payroll.approve |
| POST | /api/candidates | Add candidate | recruitment.write |
| POST | /api/documents/upload-url | Get signed upload URL | document.write |
| GET | /api/documents/{id}/download-url | Get signed download URL | document.read |
| GET | /api/reports/headcount | Headcount report | report.read |

## 9.1 Example Request and Response (proposed)

POST /api/leave/requests. Body: { leave_type_id, starts_on, ends_on, reason }. Success: 201 with { id, status: "Pending" }. Errors: 400 for invalid dates, 409 for overlapping approved leave, 403 when the caller lacks permission, 422 when balance is insufficient.

# 10. Security Architecture

- Authentication: Supabase Auth issues JWTs; sessions refresh on the client.
- RBAC: roles map to permission codes; helper functions check permissions.
- RLS: enabled on all tenant tables; policies call the same helper functions.
- Multi-tenant isolation: organization_id on every row, tested with cross-tenant cases.
- Private storage: buckets are private; downloads use short-lived signed URLs issued after a permission check.
- Server-side validation: all writes validated in the API or by database constraints.
- Secrets: stored in Vercel and Supabase environment settings, never committed to GitHub.
- Audit logging: sensitive changes recorded with actor and timestamp.
- Rate limiting and upload validation: to be implemented at the API layer; limits to be set during testing.
- Backup and recovery: rely on Supabase backup features; restore procedure to be tested before go-live.

## 10.1 Why Hiding a Button Is Not Authorization

A hidden button only changes what a normal user sees. Anyone can call the API or the database directly, so every permission must be enforced on the server or in RLS. The frontend check is a usability feature only.

## 10.2 Service-Role Key

The Supabase service-role key bypasses RLS. It must never be placed in a web, Android, or Windows client. It may be used only in trusted server code, and only where RLS cannot express the rule.

## 10.3 Common Threats and Mitigations

| Threat | Mitigation |
|---|---|
| Cross-tenant data access | RLS with organization_id checks and cross-tenant tests |
| Privilege escalation | Roles assigned only by administrators; policy tests |
| Leaked document links | Private buckets with short-lived signed URLs |
| Malicious uploads | Type and size checks; storage path generated by server |
| Key exposure | Secrets in environment settings; service-role key server-only |

# 11. User Interface and User Experience

No screenshots of the implemented application were supplied, so this chapter describes planned screens only. Any image added later must be labeled as a screenshot of the actual build, or as a conceptual wireframe.

| Screen | Planned content |
|---|---|
| Login and recovery | Email and password; password reset link |
| Admin dashboard | Organization counts, pending approvals |
| HR dashboard | Joiners, leavers, open onboarding items |
| Employee dashboard | Today's attendance, leave balance, payslips |
| Attendance | Clock button, monthly summary |
| Leave | Request form, approval queue |
| Payroll and payslips | Run list, approve action, payslip view |
| Recruitment | Pipeline board by stage |
| Documents | Folder list, upload, version history |
| Reports | Filters and tables with export |
| Android | Bottom tabs: Home, Attendance, Leave, Profile |
| Windows | Sidebar navigation with the same modules as web |

# 12. Deployment and CI/CD

## 12.1 Repository Structure (planned pnpm workspace)

apps/web (Next.js), apps/mobile (Expo), apps/desktop (Tauri), packages/shared (types and validation), supabase/ (migrations and seed data).

## 12.2 Environments

- Local: Supabase CLI for a local database, environment variables from a local file not committed.
- Preview: Vercel preview deployment per pull request, separate Supabase project.
- Production: Vercel production deployment, production Supabase project.

## 12.3 Commands

Commands to be confirmed against the actual package.json and workspace scripts. The pattern expected for this stack is: pnpm install, pnpm -r build, supabase db push for migrations, and vercel deploy from the GitHub integration.

## 12.4 GitHub Workflows

Planned: a workflow that runs install, lint, type check, and tests on each pull request; a deployment workflow for the main branch; a build workflow for Android and Windows artifacts.




**Figure 12.1. Deployment and CI/CD pipeline**

## 12.5 Android and Windows Distribution

Android: Expo build producing an installable package for internal testing. Windows: Tauri packaging producing an installer. Distribution channels to be decided with the client.

## 12.6 Backup and Restoration

Relies on Supabase backups. A restore must be rehearsed in a non-production project and the steps recorded.

# 13. Testing and Quality Assurance

All tests below are pending. No test has been executed, so no pass result may be claimed.

| ID | Scenario | Preconditions | Steps | Expected | Actual | Status |
|---|---|---|---|---|---|---|
| T-AUTH-01 | Invalid password rejected | User exists | Sign in with wrong password | Access denied | - | Pending |
| T-RLS-01 | User cannot read another tenant's employees | Two organizations seeded | Query employees with Org B token | Zero rows from Org A | - | Pending |
| T-LEAVE-01 | Overlapping leave rejected | Approved leave exists | Submit overlapping request | 409 error | - | Pending |
| T-PAY-01 | Net pay equals gross minus deductions | Known salary structure | Run payroll | Matches hand calculation | - | Pending |
| T-DOC-01 | Private file not reachable without signed URL | Document uploaded | Request raw storage path | Access denied | - | Pending |
| T-ATT-01 | Open session excluded from totals | One open clock-in | Request summary | Open session not counted | - | Pending |

Further categories to document with the same table format: unit, integration, API, end-to-end, Android, Windows, performance, and user acceptance testing.

# 14. Reports and Analytics

| Report | Contents | Access |
|---|---|---|
| Headcount | Active employees by department and branch | Manager and above, scoped |
| Attendance and overtime | Totals and exceptions for a period | Manager and HR |
| Leave | Balances and usage by type | HR and manager |
| Payroll | Run totals by component | Finance and payroll only |
| Recruitment | Candidates by stage and time to hire | Recruiter and HR |
| Performance | Review completion rates | HR and department head |
| Assets | Assigned and unassigned counts | IT |
| Audit | Sensitive change history | Auditor only |

Sensitive reports, especially payroll and audit, require explicit permissions and must be enforced in the database or server, not by hiding menu items.

# 15. Team Responsibilities and Project Plan

| Member | Responsibilities |
|---|---|
| Teja (Team Lead) | Architecture, integration, authentication and RBAC, core employee module, project presentation |
| Ashok | Attendance, shifts, leave |
| Charan | Payroll, reports, analytics |
| Kula Shekhar | UI and UX, dashboards, PowerPoint presentation, diagrams |
| Jahnavi | Recruitment, onboarding, performance |
| Afsar | SRS, database and API documentation, user manual |
| All members | Integration, testing, code review, deployment, bug fixing |

## 15.1 Phases (proposed)

| Phase | Focus | Milestone |
|---|---|---|
| 1 | Repository, Supabase project, auth, RBAC, RLS baseline | Login works and RLS tests pass |
| 2 | Employee and organization modules | Employee CRUD across three clients |
| 3 | Attendance, leave, approvals | Leave workflow end to end |
| 4 | Payroll, reports | Payroll run approved with test data |
| 5 | Recruitment, onboarding, documents, assets | Module demos |
| 6 | Testing, deployment, documentation | Submission package complete |

# 16. Advantages, Limitations, and Future Enhancements

## 16.1 Expected Advantages

- One source of truth for HR data.
- Consistent access control across clients.
- Traceable approvals and changes.

## 16.2 Current Limitations

- Policy-dependent calculations need configuration.
- Offline use not planned for first release.
- Performance not yet measured.

## 16.3 Future Work (not implemented)

- Biometric attendance devices.
- Integration with accounting systems.
- Advanced performance analytics.
- Offline mode for mobile clients.

# 17. Conclusion

The EMS proposes a single, secure platform for employee administration across web, Android, and Windows. Its design centers on one Supabase backend, database-enforced authorization, and clear module boundaries. Verification against the implemented code and execution of the test plan remain the next steps before the design can be presented as delivered.

# 18. Viva and Review Preparation

## 18.1 Two-Minute Explanation

EMS is one platform for HR work across web, Android, and Windows. All clients share a Supabase backend. Access rules live in the database through RLS, so the same rule applies to every client. Modules cover employees, attendance, leave, payroll, recruitment, and more.

## 18.2 Frequently Asked Questions

| Question | Answer |
|---|---|
| Why Supabase? | Provides PostgreSQL, Auth, Storage, and Realtime in one managed service, reducing infrastructure work for a student team. |
| What is RLS? | Row Level Security is a PostgreSQL feature that filters rows per query based on policies, so unauthorized rows are never returned. |
| Why not hide buttons? | Hiding changes the view only. The API and database must enforce permissions. |
| How is tenant data separated? | Each tenant row carries organization_id and RLS policies compare it with the caller's membership. |
| Is AI included? | No. The scope excludes AI assistants, RAG, and chatbots. |
| What is not done yet? | Full testing, measured performance, and some modules are planned. The status notes in each chapter identify these. |